

ステップ2: localStorageからタスクを復元する

ステップ1では、タスクを `localStorage` に保存する機能を実装しました。しかし、現状ではページをリロードするとタスクリストは空になってしまいます。このステップでは、ページが読み込まれた際に `localStorage` から保存されているタスクを読み込み、画面に表示する機能を実装します。

1. このステップのゴール

- ページ読み込み時に `localStorage` に保存されたタスクデータを取得し、タスクリスト (`tasks` 配列) に復元する。
- 復元されたタスクを画面に描画する。
- `localStorage` にデータがない場合でもエラーなく動作するようにする。

2. 事前知識の確認

このステップを進めるにあたり、以下の知識が必要となります。

- localStorage の基本** : `localStorage.getItem()`、`JSON.parse()` の役割を理解しておきましょう。(`localStorage` とは .md 参照)
- JavaScriptのライフサイクルイベント** : スクリプトが読み込まれた直後に処理を実行する方法。(例: スクリプトの末尾で関数を直接呼び出す)
- 条件分岐** : `localStorage` から取得したデータが存在するかどうかを確認するための `if` 文。

3. 具体的な手順: `script.js` の変更

`script.js` に、ページ読み込み時に `localStorage` からタスクを読み込む `loadTasks` 関数を作成し、スクリプトの初期化处理の中で呼び出すようにします。

3.1. `loadTasks` 関数の作成

`localStorage` からタスクデータを読み込み、`tasks` 配列を更新し、画面を再描画する `loadTasks` 関数を作成します。

```
// script.js の適切な場所（例：saveTasks 関数の後）

// localStorageからタスクを読み込む関数
function loadTasks() {
  const storedTasks = localStorage.getItem(STORAGE_KEY);
  if (storedTasks) {
    try { // ★追加: JSON.parseのエラーハンドリング
      tasks = JSON.parse(storedTasks);
      console.log('localStorageからタスクを読み込みました:', tasks);
    } catch (e) {
      console.error('localStorageからタスクを読み込めませんでした。データが破損している可能性があります。', e);
      tasks = []; // エラー時は空のタスクリストとして扱う
    }
  } else {
    console.log('localStorageに保存されたタスクはありません。');
  }
  renderTasks(); // 読み込んだタスク（または空のリスト）を描画
}
```

コード解説:

- `localStorage.getItem(STORAGE_KEY)` : ステップ1で保存時に使用したキー（`STORAGE_KEY`）を使って、`localStorage` からデータを取得します。
- `if (storedTasks)` : `localStorage` にデータが保存されていれば、`storedTasks` にはJSON文字列が入ります。データがなければ `null` になります。この条件分岐で、データが存在する場合のみ処理を行います。
- `try...catch` : `JSON.parse()` は、引数が有効なJSON文字列でない場合にエラーを投げる可能性があります。例えば、`localStorage` のデータが何らかの理由で破損していた場合などです。`try...catch` で囲むことで、エラーが発生してもプログラム全体が停止するのを防ぎ、エラーメッセージをコンソールに出力して、`tasks` 配列を空にするなどの代替処理を行うことができます。
- `tasks = JSON.parse(storedTasks);` : 取得したJSON文字列を `JSON.parse()` を使ってJavaScriptの配列（オブジェクトの配列）に戻し、グローバル変数 `tasks` に代入します。
- `console.log()` : 読み込み処理の状況をコンソールに出力します。
- `renderTasks()` : `localStorage` からタスクを読み込んだ後（または何も読み込まなかった後）、現在の `tasks` 配列の内容に基づいて画面のタスク一覧を更新します。これにより、保存されていたタスクが画面に表示されます。

3.2. データフローの図解 (イメージ)

ページ読み込み時にタスクが復元されるまでのデータの流れを簡単に示すと以下のようになります。

```
graph TD
  A[ページ読み込み開始] --> B[loadTasks関数実行];
  B --> C[localStorageからデータを取得 (キー: todoTasks)];
  C -- データあり --> D[JSON文字列をパース];
  D -- 成功 --> E[tasks配列を更新];
```

```
D -- パース失敗（エラー） --> F[エラーメッセージ表示、tasks配列を空に];
C -- データなし --> G[tasks配列は変更なし（初期状態のまま）];
E --> H[renderTasks関数実行];
F --> H;
G --> H;
H --> I[画面のリストを更新/表示];
```

3.3. 初期化処理の変更

スクリプトが読み込まれたときに `loadTasks` 関数が実行されるように、`script.js` の末尾にある初期化処理を変更します。

以前は `renderTasks();` のみを呼び出していましたが、これを `loadTasks();` の呼び出しに置き換えます。`loadTasks` 関数内で `renderTasks` が呼び出されるため、二重に呼び出す必要はありません。

```
// script.js の末尾

// イベントリスナー
addTaskButton.addEventListener('click', addTask);

taskInput.addEventListener('keypress', function(event) {
  if (event.key === 'Enter') {
    addTask();
  }
});

// 初期化処理：ページ読み込み時にlocalStorageからタスクを読み込む
loadTasks(); // ★変更点：renderTasks() から loadTasks() の呼び出しに変更
```

コード解説:

- `loadTasks();` : この行がスクリプトの最後に実行されることで、ページが完全に読み込まれ、JavaScriptが実行可能になった直後に `loadTasks` 関数が呼び出されます。これにより、保存されていたタスクがページ表示と同時に復元されます。

4. 演習課題

- 解答例/step1_save/script.js をコピーして 演習用ベースコード/script.js として上書きするか、ステップ1の変更が適用された `script.js` を用意します。
- 上記の「3.1. `loadTasks` 関数の作成」と「3.2. 初期化処理の変更」で示された変更を `script.js` に加えてください。
 - `loadTasks` 関数を新しく作成しましょう。
 - スクリプト末尾の `renderTasks();` の呼び出しを `loadTasks();` に変更しましょう。
- `index.html` をブラウザで開きます。

- まず、タスクをいくつか追加し、ページをリロードしてみてください。タスクが保持されて表示されるはずです。
- 次に、開発者ツールで `localStorage` の `todoTasks` を一度削除（またはキー名を変更）してからページをリロードしてみてください。タスクが表示されず、コンソールに「localStorageに保存されたタスクはありません。」と表示されることを確認します。

5. 動作確認の方法

- 1. タスクの追加とリロード** : `index.html` をブラウザで開き、いくつかのタスクを追加します。その後、ブラウザの更新ボタンを押すか、`F5` キーでページをリロードします。
 - **期待される動作** : リロード後も、追加したタスクが画面に表示されていること。
- 2. localStorage の確認** : 開発者ツールの「Application」タブ（または「ストレージ」タブ）で、`localStorage` の `todoTasks` にデータが保存されていることを確認します。
- 3. localStorage のクリアとリロード** : 開発者ツールで `todoTasks` の項目を右クリックして「削除」するか、コンソールで `localStorage.removeItem('todoTasks')` を実行してデータを削除します。その後、ページをリロードします。
 - **期待される動作** : タスクリストが空の状態が表示され、コンソールには「localStorageに保存されたタスクはありません。」というメッセージが表示されること。

よくあるエラーと対処法:

- **リロードしてもタスクが表示されない** : `loadTasks()` 関数が正しく呼び出されているか、`STORAGE_KEY` の名前が保存時と読み込み時で一致しているか確認してください。`JSON.parse()` が正しく行われているかも確認ポイントです。開発者ツールのコンソールにエラーメッセージが出ていないか確認しましょう。
- **JSON.parse() でエラーが出る (コンソールに "SyntaxError" など)** : `localStorage` に保存されているデータが正しいJSON形式でない可能性があります。ステップ1の保存処理 (`JSON.stringify()`) が正しく機能しているか、または手動で `localStorage` の値を不正な形式に変更していないか確認してください。上記の `try...catch` を実装していれば、エラーメッセージがコンソールに出力されるはずですが。

6. Q&A

Q1: `loadTasks()` はいつ呼び出すのが最適ですか？

A1: `loadTasks()` は、ページのHTML構造（タスクリストを表示する要素など）が読み込まれ、JavaScriptが操作できるようになった後、かつユーザーがページを操作し始める前に呼び出すのが一般的です。`script.js` を `<body>` タグの末尾で読み込んでいる場合、スクリプトの最後で `loadTasks()` を呼び出せば、これらの条件を満たせます。

Q2: `localStorage.getItem()` が返す値は何ですか？データがない場合は？

A2: `localStorage.getItem(key)` は、指定されたキーに対応する値（文字列）を返します。もしキーが存在しない場合、`null` を返します。そのため、`if (storedTasks)` のような条件分岐で、データが存在するかどうかを確認できます。

Q3: `JSON.parse()` が失敗する具体的な例は？

A3: 例えば、`localStorage` に手動で `{"text":"test"}` のような、閉じ括弧が足りない不正なJSON文字列を保存してしまった場合、`JSON.parse("{\"text\":\"test\"")` は `SyntaxError` を引き起こします。また、JSONの仕様に準拠していない形式（例: シングルクォートを使った文字列 `'{"text":"test"}'` や、キーがダブルクォートで囲まれていない `{text:"test"}` など）もパースエラーの原因となります。

これで、タスクの保存と読み込みの基本的な機能が完成しました。次のステップでは、タスクの完了状態も永続化できるように機能を拡張していきます。